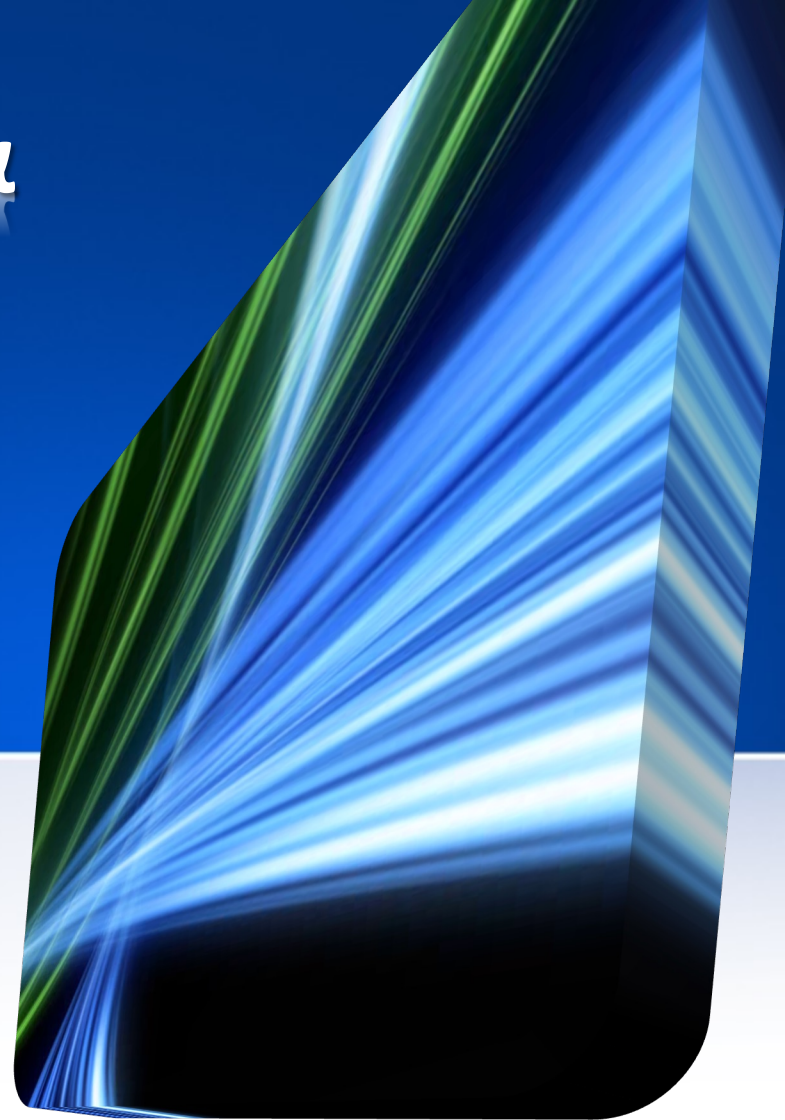


Λειτουργικά Συστήματα

6ο εξάμηνο ΣΗΜΜΥ

Ακ. έτος 2025-2026

Εργαστηριακή Άσκηση 2



Θεωρία Εργ. Άσκησης 2

Σήματα



Σήμα (**signal**) είναι ένας τρόπος επικοινωνίας των διεργασιών.

Είναι μια ειδοποίηση που στέλνεται σε μια διεργασία προκειμένου να την ενημερώσει για κάποιο **γεγονός**.

Η διεργασία που λαμβάνει ένα σήμα **διακόπτει** την εκτέλεση της και αναγκάζεται να χειριστεί το σήμα **άμεσα**.

Θεωρία Εργ. Άσκησης 2

Σήματα

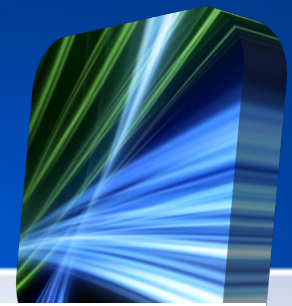


Κάθε σήμα έχει έναν ακέραιο αριθμό που το αντιπροσωπεύει (1,2, ...), καθώς επίσης και ένα συμβολικό όνομα που καθορίζεται συνήθως στο αρχείο `/usr/include/signal.h`

Με την εντολή "**kill -l**" βλέπουμε τον κατάλογο σημάτων που υποστηρίζονται από το σύστημά μας.

Θεωρία Εργ. Άσκησης 2

Λίστα Σημάτων



Signal	Value	Action	Comment
SIGHUP	1	Term	Hangup detected on controlling terminal or death of controlling process
SIGINT	2	Term	Interrupt from keyboard
SIGQUIT	3	Core	Quit from keyboard
SIGILL	4	Core	Illegal Instruction
SIGABRT	6	Core	Abort signal from <code>abort(3)</code>
SIGFPE	8	Core	Floating point exception
SIGKILL	9	Term	Kill signal
SIGSEGV	11	Core	Invalid memory reference
SIGPIPE	13	Term	Broken pipe: write to pipe with no readers
SIGALRM	14	Term	Timer signal from <code>alarm(2)</code>
SIGTERM	15	Term	Termination signal
SIGUSR1	30,10,16	Term	User-defined signal 1
SIGUSR2	31,12,17	Term	User-defined signal 2
SIGCHLD	20,17,18	Ign	Child stopped or terminated
SIGCONT	19,18,25	Cont	Continue if stopped
SIGSTOP	17,19,23	Stop	Stop process
SIGTSTP	18,20,24	Stop	Stop typed at terminal
SIGTTIN	21,21,26	Stop	Terminal input for background process
SIGTTOU	22,22,27	Stop	Terminal output for background process

Θεωρία Εργ. Άσκησης 2

Σήματα



Κάθε σήμα μπορεί να έχει έναν **χειριστή σήματος** (handler), ο οποίος είναι μια συνάρτηση που εκτελείται όταν η διεργασία λαμβάνει το συγκεκριμένο σήμα. Η συνάρτηση αυτή καλείται ασύγχρονα.

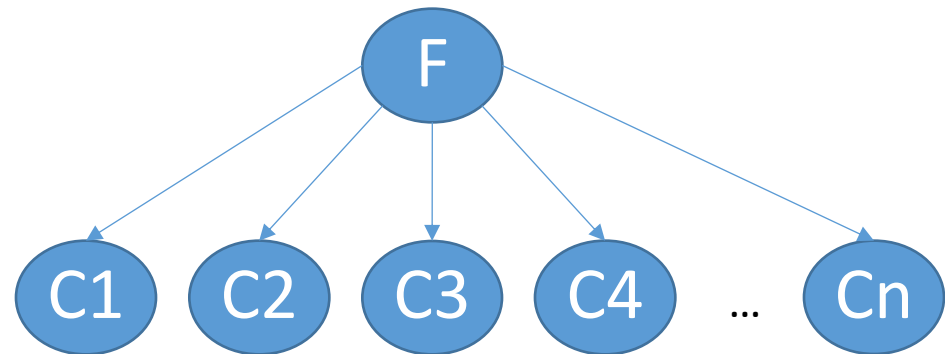
Όταν στέλνεται το σήμα σε μια διεργασία, το ΛΣ σταματά την εκτέλεση της διεργασίας, και "την αναγκάζει" να καλέσει την αντίστοιχη συνάρτηση χειρισμού αυτού του σήματος (handler).

Όταν η συνάρτηση ολοκληρώσει την εκτέλεσή της, η διεργασία συνεχίζει την εκτέλεση από το σημείο που βρισκόταν αμέσως πριν παραληφθεί το σήμα.

Εργαστηριακή Άσκηση 2



Να γραφτεί πρόγραμμα C σε περιβάλλον Linux, στο οποίο η διεργασία-πατέρας (F) δημιουργεί n διεργασίες-παιδιά (C1, C2, C3, C4 ... Cn) σύμφωνα με το παρακάτω δέντρο διεργασιών:



Ο χρήστης θα μπορεί να στέλνει σήματα **SIGUSR1**, **SIGUSR2**, **SIGTERM** στην διεργασία πατέρα και στις διεργασίες παιδιά. Ο χειρισμός αυτών των σημάτων θα εξηγηθεί αναλυτικά στις επόμενες διαφάνειες

Εργαστηριακή Άσκηση 2



Παράδειγμα εκτέλεσης προγράμματος

`./assignment2 n`

Ο πατέρας δημιουργεί n διεργασίες. Κάθε μια διεργασία παιδί θα "διαχειρίζεται" μια ακέραη μεταβλητή. Ο πατέρας δέχεται τον αριθμό των διεργασιών κατά την εκτέλεση του προγράμματος.

Εργαστηριακή Άσκηση 2



Αρχικά, η διεργασία πατέρας δημιουργεί όλες τις διεργασίες παιδιά και τυπώνει το δέντρο διεργασιών που έχει δημιουργηθεί ώστε να φαίνονται τα PID τους.

Εργαστηριακή Άσκηση 2



Οι διεργασίες παιδιά κάθε 10 δευτερόλεπτα τυπώνουν την τιμή της μεταβλητής τους (με alarm) και τον χρόνο που εκτελούνται

Εργαστηριακή Άσκηση 2



Οι διεργασίες παιδιά

- όταν λαμβάνουν σήμα SIGUSR1 αυξάνουν την μεταβλητή κατά 1
- όταν λαμβάνουν σήμα SIGUSR2 μειώνουν την μεταβλητή κατά 1.
- όταν λαμβάνουν σήμα SIGTERM τερματίζουν

Οι διεργασίες παιδιά μπορούν να λάβουν σήμα είτε από τον χρήστη από το terminal είτε από τον πατέρα

Εργαστηριακή Άσκηση 2



Η διεργασία πατέρας

- όταν λάβει σήμα SIGUSR1, στέλνει SIGUSR1 σε όλα τα παιδιά.
- όταν λάβει σήμα SIGUSR2, στέλνει SIGUSR2 σε όλα τα παιδιά.
- όταν λάβει σήμα SIGTERM, στέλνει SIGTERM σε όλα τα παιδιά.
- φροντίζει επίσης όλα τα παιδιά να είναι υγιή, οπότε ελέγχει για σήματα SIGCHLD και (α) αν κάποιο παιδί έχει σκοτωθεί, το κάνει wait και φτιάχνει καινούριο στη θέση του (β) αν κάποιο παιδί έχει σταματήσει, το κάνει resume. Θα πρέπει να υπάρχουν πάντα N παιδιά ενεργά.

Η διεργασία πατέρας μπορεί να λάβει σήμα από τον χρήστη από το terminal.

Εργαστηριακή Άσκηση 2

Απαραίτητες Υποδείξεις



- Έλεγχος ορθότητας ορισμάτων (`argc`, `argv`) και εκτύπωση κατάλληλου μηνύματος
- Έλεγχος σφαλμάτων για κάθε κλήση συστήματος
- Ο κώδικας των παιδιών είναι σε ξεχωριστό αρχείο `child.c`.
Να γίνει χρήση `execv`

Εργαστηριακή Άσκηση 2



Μια διεργασία μπορεί να στείλει ένα σήμα σε μια άλλη, με την κλήση **kill(pid, SIGNAL)** η οποία στέλνει το σήμα με όνομα **SIGNAL** στη διεργασία με process id ίσο με το όρισμα **pid**.

Για παράδειγμα με την εκτέλεση της εντολή **kill(3247, SIGCONT)** στέλνεται το σήμα **SIGCONT** στην διεργασία με pid = 3247

Εργαστηριακή Άσκηση 2



Η εντολή αποστολής ενός σήματος από terminal είναι

kill -*SIGNAL* pid

όπου ***SIGNAL*** το όνομα του σήματος που θέλουμε να αποσταλεί και **pid** το process id της διεργασίας στην οποία θέλουμε να στείλουμε το σήμα. Για παράδειγμα η εντολή **kill -*SIGUSR1* 3216** στέλνει το σήμα ***SIGUSR1*** στην διεργασία με pid = 3216

Εργαστηριακή Άσκηση 2



Η **waitpid()** χρησιμοποιείται για να περιμένει η καλούσα διεργασία αλλαγές κατάστασης σε μια διεργασία παιδί της και να λάβει πληροφορίες σχετικά με την διεργασία της οποίας η κατάσταση έχει αλλάξει

Εργαστηριακή Άσκηση 2



Ορισμός **waitpid()**

pid_t waitpid(pid_t pid, int *wstatus, int options)

- Το πεδίο **pid** ορίζει το pid της διεργασίας που περιμένουμε να αλλάξει κατάσταση (με τιμή ίση με -1 περιμένουμε οποιαδήποτε διεργασία παιδί)
- Το πεδίο **wstatus** είναι ο δείκτης στην μεταβλητή στην οποία μπορούμε να αποθηκεύσουμε την κατάσταση της διεργασίας παιδί
- Το πεδίο **options** ορίζει επιπλέον επιλογές

Περισσότερες πληροφορίες

<http://www.man7.org/linux/man-pages/man2/waitpid.2.html>

Εργαστηριακή Άσκηση 2



Μια διεργασία μπορεί να ορίσει τον χειριστή (handler) κάθε σήματος που θα λάβει με τις συναρτήσεις:

- **sigaction()** (προτείνεται)
- **signal()**

Η κλήση **signal(SIGNAL, handler)** καθορίζει ότι όταν η καλούσα διεργασία λάβει σήμα με όνομα **SIGNAL** θα εκτελέσει τη συνάρτηση χειρισμού σήματος **handler**.

Εργαστηριακή Άσκηση 2



Η κλήση συστήματος `sigaction()` χρησιμοποιείται για να αλλάξει τη δράση (action) που θα γίνει από μια διεργασία κατά την λήψη συγκεκριμένου σήματος.

```
int sigaction(int signum, const struct sigaction *act, struct sigaction *oldact);
```

- Η παράμετρος **signum** καθορίζει το σήμα και μπορεί να είναι οποιοδήποτε έγκυρο σήμα εκτός από SIGKILL και SIGSTOP
- Εάν η παράμετρος **act** είναι not NULL τότε θέτει το νέο sigaction για το δεδομένο signal
- Εάν η παράμετρος **oldact** είναι not NULL τότε εκεί αποθηκεύεται η προηγούμενη ορισμένη δράση για το δεδομένο signal

Εργαστηριακή Άσκηση 2



Παράδειγμα βασικής χρήσης **sigaction()**

struct sigaction action;	//δήλωση δομής struct sigaction
action.sa_handler = myfunction;	//ορισμός ονόματος συνάρτησης χειρισμού
sigaction(SIGUSR1, &action, NULL);	//ορισμός χειριστή (myfunction) για το σήμα SIGUSR1

Αποτέλεσμα: όταν η διεργασία λάβει το σήμα **SIGUSR1** θα διακόψει την εκτέλεση της και θα καλέσει την συνάρτηση **myfunction()**

Περισσότερες πληροφορίες

<http://man7.org/linux/man-pages/man2/sigaction.2.html>

Εργαστηριακή Άσκηση 2



Οι διεργασίες μπορούν να ορίσουν να εκτελούν μια εργασία μετά την πάροδο συγκεκριμένου χρόνου. Για να το πετύχουν αυτό οι διεργασίες μπορούν να κάνουν κλήση της συνάρτησης **alarm()**

Η συνάρτηση **alarm(unsigned int seconds)** θα προκαλέσει το σύστημα να παράγει ένα σήμα **SIGALRM**, προς τη διεργασία που την κάλεσε, μετά την παρέλευση του αριθμού των δευτερόλεπτων πραγματικού χρόνου που καθορίζονται από την παράμετρο **seconds**.

Χρήσιμη αναφορά



<http://man7.org/linux/man-pages/man7/signal.7.html>